



SQL DDL: Criando o Esquema

Aula 13 · Bloco 4 — SQL Básico

O papel vira um arquivo que qualquer PostgreSQL executa

Nesta aula

1. Do **esquema** ao **script**
2. `CREATE TABLE` e os **tipos**
3. `NOT NULL`, `UNIQUE`, `CHECK`
4. `PRIMARY KEY` e `FOREIGN KEY`
5. `ON DELETE` e `ON UPDATE`
6. `ALTER` e `DROP`
7. Os **erros** do PostgreSQL

A tradução é direta

No papel

OBRA(isbn, titulo, ano)

ano entre 1450 e hoje

Em SQL

```
CREATE TABLE obra (  
    isbn    VARCHAR(17) NOT NULL,  
    titulo  VARCHAR(150) NOT NULL,  
    ano     INTEGER NOT NULL,  
    CONSTRAINT obra_pk PRIMARY KEY (isbn),  
    CONSTRAINT obra_ano_ck  
        CHECK (ano BETWEEN 1450 AND 2100)  
);
```

Nome → nome · atributo → coluna · domínio → tipo + **CHECK** · sublinhado → **PRIMARY KEY**

O script fica no repositório e roda do zero

Sempre.

Nada de banco construído a cliques
que ninguém sabe reproduzir.

Se o seu esquema não cabe
num arquivo versionado,
ele não existe.

As três restrições da Aula 04, agora escritas

Em SQL	Garante	Qual integridade
NOT NULL	a coluna nunca fica vazia	domínio
UNIQUE	o valor não se repete	a chave alternativa (Aula 02)
CHECK (...)	a condição vale em toda linha	domínio e semântica

```
CONSTRAINT usuario_categoria_ck CHECK (categoria IN ('aluno','professor','servidor')),  
CONSTRAINT emprestimo_prazo_ck CHECK (data_prevista >= data_retirada)
```

Dê nome às suas restrições

Sem nome, o erro que chega ao usuário
fala de `usuario_categoria_check`.

Com nome escolhido por você,
ele fala de uma **regra do seu modelo**.

É a diferença entre
um log legível e um enigma.

PRIMARY KEY e FOREIGN KEY

```
CONSTRAINT telefone_pk PRIMARY KEY (matricula, numero),  
  
CONSTRAINT emprestimo_usuario_fk FOREIGN KEY (matricula)  
    REFERENCES usuario (matricula)
```

PRIMARY KEY já implica **NOT NULL** e **UNIQUE** — a integridade de entidade inteira, numa palavra.

E aceita mais de uma coluna: é a chave composta da Aula 02.

O banco recusando

```
ERROR:  insert or update on table "emprestimo" violates foreign key  
        constraint "emprestimo_usuario_fk"  
DETAIL:  Key (matricula)=(999999999) is not present in table "usuario".
```

A mensagem diz **a tabela, a restrição, a coluna e o valor culpado**.

O PostgreSQL é bom nisso — e a maioria das pessoas desiste de ler no **ERROR:**.

ON DELETE : as ações da Aula 04, com sintaxe

Em SQL

Ao apagar a linha referenciada

ON DELETE RESTRICT

recusa a operação — **é o padrão**

ON DELETE CASCADE

apaga junto as dependentes

ON DELETE SET NULL

esvazia a FK (exige coluna opcional)

```
-- sem o usuário, o telefone é um número solto  
REFERENCES usuario (matricula) ON DELETE CASCADE
```

Cascata só onde a linha não faz sentido sozinha

Um **DELETE** de **uma linha** pode apagar
exemplares, empréstimos,
renovações e multas.

Em silêncio. Sem pergunta.

O comando que causou isso
tinha uma linha.

ALTER e DROP

```
ALTER TABLE usuario ADD COLUMN ativo BOOLEAN NOT NULL DEFAULT TRUE;  
ALTER TABLE usuario ADD CONSTRAINT usuario_email_ck CHECK (email LIKE '%@%');  
  
DROP TABLE IF EXISTS multa;           -- a ordem dos DROP é a INVERSA dos CREATE  
DROP TABLE IF EXISTS emprestimo;
```

⚠ **ADD COLUMN ... NOT NULL sem DEFAULT falha** se a tabela já tiver linhas — o banco não sabe o que pôr nas existentes.

Os três erros que você vai ver

```
ERROR: duplicate key value violates unique constraint "usuario_pk"  
→ você rodou a carga duas vezes
```

```
ERROR: new row violates check constraint "usuario_categoria_ck"  
DETAIL: Failing row contains (... , estagiario, ...)  
→ o domínio recusou. O culpado está no DETAIL
```

```
ERROR: relation "aluno" does not exist  
→ banco errado, ou você usou aspas duplas no nome
```

Nunca use aspas duplas em nome de tabela ou coluna.

Exercícios da aula

Na pasta `aula-13/` :

1. `ex01.sql` — o DDL do **seu** modelo, rodando do zero **duas vezes seguidas**;
2. `ex02.sql` — três `CHECK` do seu minimundo, um comparando duas colunas;
3. `ex03.sql` — três `INSERT` que **devem falhar**, com as mensagens coladas;
4. `ex04.sql` — dois `ALTER TABLE` num banco que já tem dados;
5. **Desafio**  `ex05.sql` — quebre uma ação referencial da Biblioteca e mostre o estrago.

Próxima aula

Aula 14 — SQL DML e o **SELECT** simples

Pôr dado dentro,
tirar dado de dentro,
e a armadilha do vazio.